LAPORAN PRAKTIKUM
ALGORITMA DAN PEMROGRAMAN 2
MODUL 4
“PROSEDUR”

DISUSUN OLEH:

Dharma Chandra Viriya

109082500052

S1 IF-13-02

DOSEN:

Wahyu Andi Saputra, S.Pd., M.Eng.

PROGRAM STUDI S1 TEKNIK INFORMATIKA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2024/2025

DASAR TEORI

CONTOH SOAL

1. Latihan1

Source Code:

Output:

Deskripsi Program:

SOAL LATIHAN

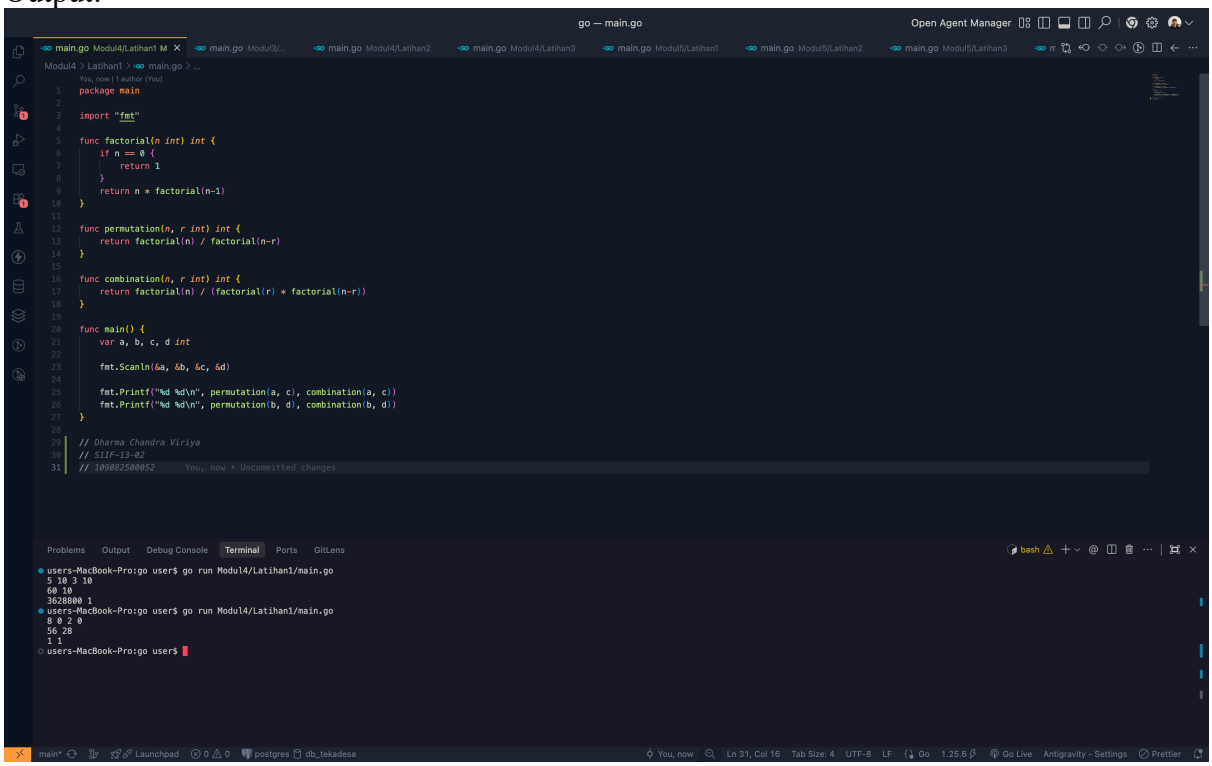
Statement prosedur

1. **Latihan1**

Source Code:

```
package main

import "fmt"

func factorial(n int) int {
	if n == 0 {
		return 1
	}
	return n * factorial(n-1)
}

func permutation(n, r int) int {
	return factorial(n) / factorial(n-r)
}

func combination(n, r int) int {
	return factorial(n) / (factorial(r) * factorial(n-r))
}

func main() {
	var a, b, c, d int

	fmt.Scanln(&a, &b, &c, &d)

	fmt.Printf("%d %d\n", permutation(a, c), combination(a, c))
	fmt.Printf("%d %d\n", permutation(b, d), combination(b, d))
}
```

Output:



Deskripsi Program:
Program Go tersebut dipakai untuk menghitung susunan dan perpaduan dari beberapa angka yang diberikan oleh pemakai. Dalam program ini diberlakukan konsep faktorial sebagai metode utama pada perhitungan matematika ilmu kombinatorika. Pemakai diminta untuk memasukkan empat bilangan bulat, setelah itu program akan menghitung hasil susunan dan perpaduan dari bilangan tersebut. Hasil dari perhitungan ini kemudian diperlihatkan dalam dua baris keluaran.

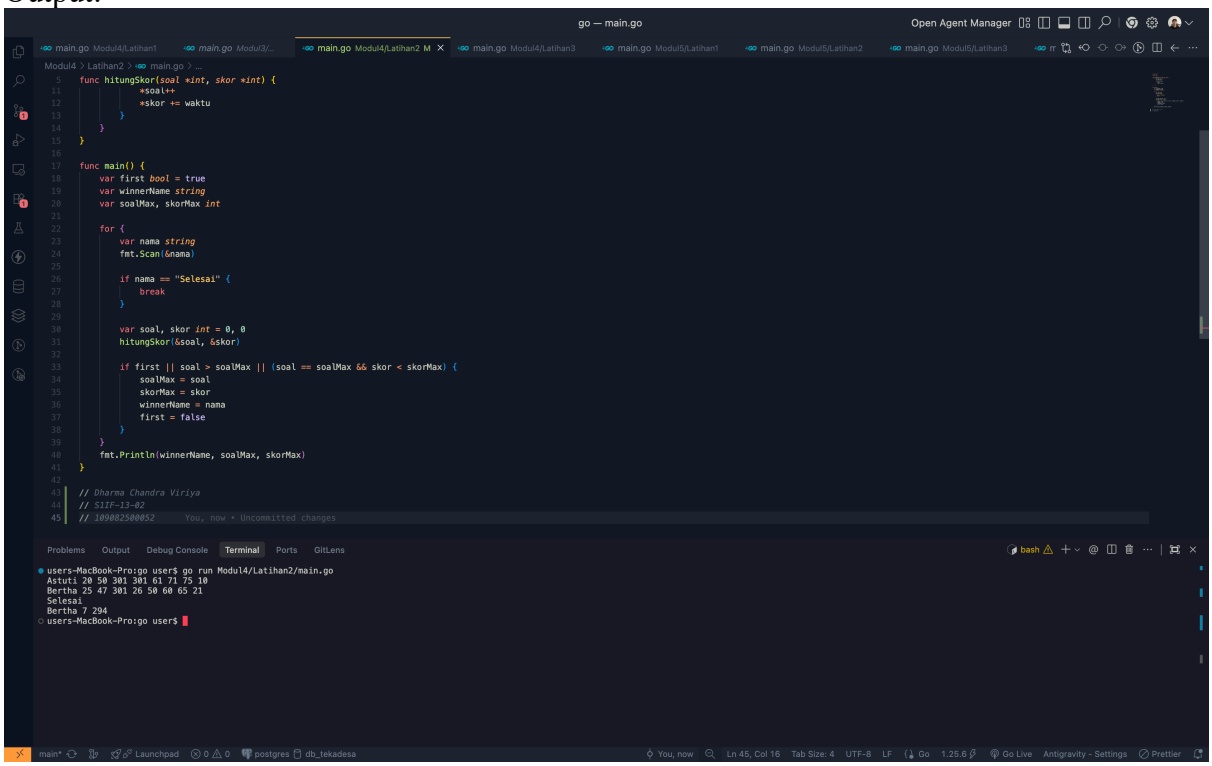
Secara teknis, program ini membuat suatu fungsi `factorial` yang menerapkan pendekatan rekursif, di mana fungsi akan memanggil dirinya lagi hingga mencapai kondisi dasar saat nilai `n` menjadi nol. Nilai faktorial ini kemudian dipakai oleh fungsi `permutation` untuk menghitung rumus (P(n,r) = n! / (n-r)!) dan oleh fungsi `combination` untuk menghitung rumus (C(n,r) = n! / (r! (n-r)!)). Dalam fungsi `main`, empat angka masukan dibaca memakai `fmt. Scanln`, lalu hasil dari fungsi susunan dan perpaduan dipanggil dan diperlihatkan dengan `fmt. Printf`. Struktur program yang sederhana ini menunjukkan bagaimana fungsi dapat dimanfaatkan lagi untuk melakukan berbagai penghitungan matematika secara efektif.

2. **Latihan2**

Source Code:

```
package main

import "fmt"

func hitungSkor(soal *int, skor *int) {
	for i := 0; i < 8; i++ {
		var waktu int
		fmt.Scan(&waktu)

		if waktu != 301 {
			*soal++
			*skor += waktu
		}
	}
}

func main() {
	var first bool = true
	var winnerName string
	var soalMax, skorMax int

	for {
		var nama string
		fmt.Scan(&nama)

		if nama == "Selesai" {
			break
		}

		var soal, skor int = 0, 0
		hitungSkor(&soal, &skor)

		if first || soal > soalMax || (soal == soalMax && skor < skorMax) {
			soalMax = soal
			skorMax = skor
			winnerName = nama
			first = false
		}
	}
	fmt.Println(winnerName, soalMax, skorMax)
}
```

Output:



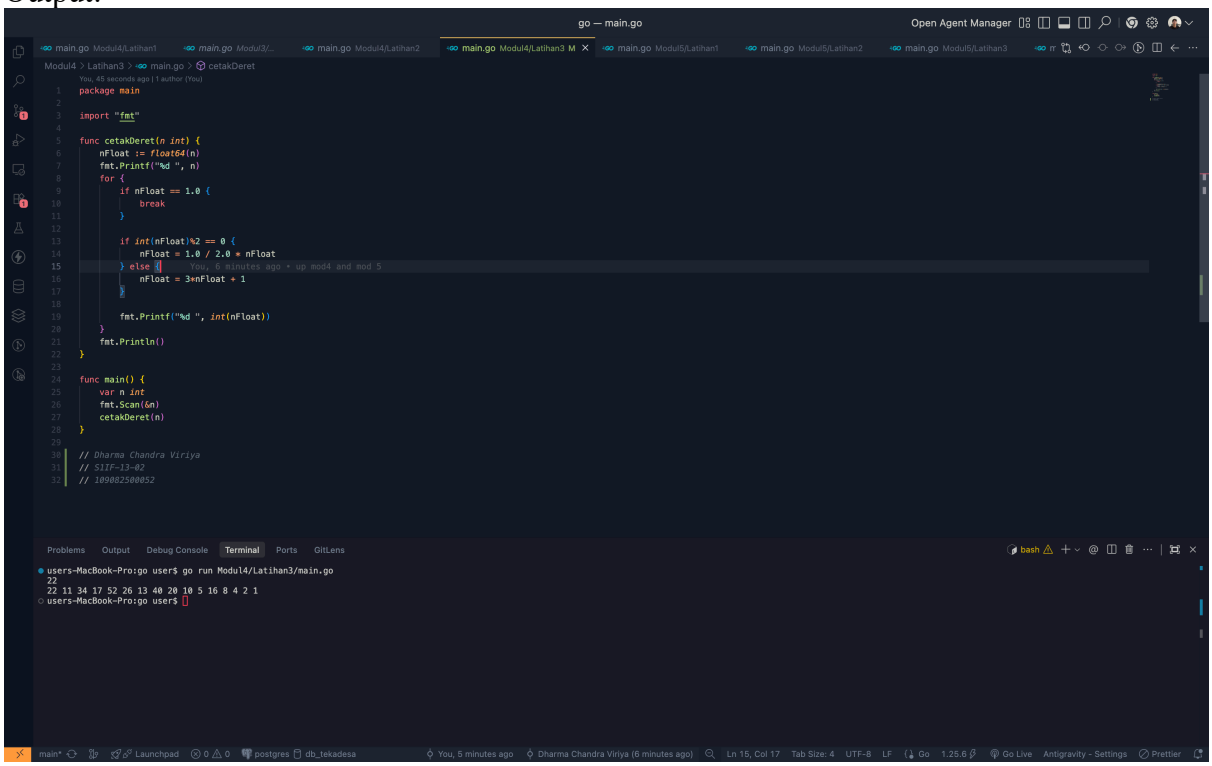
Deskripsi Program:
Program ini terdiri dari fungsi `main` dan `hitungSkor`. Fungsi `hitungSkor` menggunakan parameter pointer (`*int`) untuk mengubah langsung nilai `soal` dan `skor` dari fungsi pemanggil. Di dalamnya dilakukan perulangan sebanyak 8 kali untuk membaca input waktu tiap soal. Jika nilai waktu tidak sama dengan 301, maka pointer `soal` di-increment (`*soal++`) dan pointer `skor` ditambah dengan waktu tersebut (`*skor += waktu`). Pada fungsi `main`, program melakukan perulangan input nama peserta hingga ditemukan string `"Selesai"`. Untuk setiap peserta, variabel `soal` dan `skor` diinisialisasi lalu dikirim sebagai alamat ke fungsi `hitungSkor`. Hasilnya kemudian dibandingkan dengan nilai maksimum sebelumnya menggunakan kondisi logika: peserta pertama langsung menjadi acuan, selanjutnya akan diganti jika jumlah soal lebih besar atau jika jumlah soal sama tetapi skor lebih kecil. Nilai terbaik disimpan dalam `soalMax`, `skorMax`, dan `winnerName`, lalu di akhir program ditampilkan menggunakan `fmt.Println`.

3. **Latihan3**

Source Code:

```
package main

import "fmt"

func cetakDeret(n int) {
	nFloat := float64(n)
	fmt.Printf("%d ", n)
	for {
		if nFloat == 1.0 {
			break
		}

		if int(nFloat)%2 == 0 {
			nFloat = 1.0 / 2.0 * nFloat
		} else {
			nFloat = 3*nFloat + 1
		}

		fmt.Printf("%d ", int(nFloat))
	}
	fmt.Println()
}

func main() {
	var n int
	fmt.Scan(&n)
	cetakDeret(n)
}
```

Output:



Deskripsi Program:

Program ini berfungsi untuk mencetak deret bilangan berdasarkan aturan *Collatz sequence*. Pada fungsi `main`, program membaca sebuah bilangan bulat `n` dari input, lalu memanggil fungsi `cetakDeret` dengan nilai tersebut. Di dalam fungsi `cetakDeret`, nilai `n` dikonversi ke tipe `float64` agar bisa digunakan dalam operasi pembagian yang menghasilkan bilangan desimal. Program kemudian mencetak nilai awal `n`, lalu masuk ke perulangan tak hingga (`for`) yang akan berhenti jika nilai sudah menjadi 1. Pada setiap iterasi, dilakukan pengecekan apakah nilai saat ini genap atau ganjil dengan `int(nFloat)%2`. Jika genap, nilai dibagi 2, sedangkan jika ganjil, nilai diubah menjadi `3n + 1`. Setiap hasil perhitungan kemudian dicetak sebagai bilangan bulat. Proses ini terus berlangsung hingga mencapai 1, lalu program berhenti dan mencetak baris baru.

DAFTAR PUSTAKA